2026-06-24 — Search "no results" + "can't go back/ interrupt" — root-cause investigation

Operator (2026-06-24, released apps 1.3.11-1072 + api-app 0.2.11-18): "search and UX around it does not work — we still do not get any results; onboarded with just YTS, same issue; once we enter search and start it we can't go back or interrupt it."

Investigation per §6.T.1 (Reproduction-Before-Fix) + constitution §11.4.152 (Crashlytics-monitoring

- systematic-debug + regression-test-coverage). systematic-debugging Phase 1.

Decisive field evidence (Firebase Crashlytics — the §6.AC telemetry shipped in 1071/1072)

Client release app `1:815513478335:android:456475e2ef4039d8cfd20a`, top NON_FATAL issue `9d4ad2f4d1a8b8697b1506402e045b81`, sample event `…_2233309019041036508` (2026-06-24T07:39:36Z):

- Exception: **`java.net.SocketTimeoutException: timeout`** at `okhttp3.internal.http2.Http2Stream.takeHeaders` → `Http2ExchangeCodec.readResponseHeaders` (the request was SENT through the full OkHttp chain incl. `AuthInterceptor.kt:43`; only the **response headers never arrived** → HTTP/2 read-headers timeout).
- customKeys (from `SearchResultViewModel.recordProviderFailure`): `feature=search`, `operation=streamMultiSearch`, `provider=yts`, `screen=search_result`, `error_message=timeout`, `build_type=release`, `version=1.3.11 (1072)`. Device HUAWEI TXZ-W09 / Android 12 / LANDSCAPE.
- api-app (engine) Crashlytics …:`d57b960e`… over the same window: **NO issues** — the engine is NOT crashing; it is running but **not responding** to the search within the client's timeout.

Root cause A — "no results": engine response exceeds the client read timeout

- Client `@Named("lan")` OkHttpClient: `readTimeout = 30s` (`core/network/impl/.../NetworkModule.kt:174`).
- Engine yts client: `DefaultTimeout = 20s` (whole client), `perAttemptTimeout = 8s` per mirror (`lava-api-go/internal/provider/curated/yts/client.go:46,51`). Failover across N mirrors × 8s (+ §11.4.85 bounded retries) can exceed the client's 30s — especially when yts.mx is slow/ unreachable from the phone's network. The engine's GET /v1/yts/search does NOT stream/return- partial; the client waits for the whole search → `readResponseHeaders` times out at 30s.
- The §6.AC ProviderFailure → `handleStreamEnd()` Error+Retry fix (cfe838bc) DOES fire — but only AFTER the 30s hang, so the user-visible experience is "stuck 30s, then no results."

Fix direction: bound the engine's TOTAL search-handler time to well under the client's read timeout (a request-context deadline, e.g. ~15s) so the engine ALWAYS responds in time with results OR a fast "provider timed out/unreachable" error — never leaving the client to time out.

Root cause B — "can't go back / interrupt": hung search, no cancel

- `observeStreamMultiSearch` fires in `container.onCreate` and blocks on `sdk.streamMultiSearch(...) .collect{}` for up to 30s. There is NO cancel affordance and `onBackClick()` only posts a `Back` side effect — it does NOT cancel the in-flight search job. (No `BackHandler` swallows back in the search screens, so it is not a back-swallow.) The exact UI block (Orbit intent serialization vs a blocking loading state) is confirmed by the reproduction test below.

Fix direction: track the streamMultiSearch job and cancel it on `BackClick` (and on a new search); make the in-flight search interruptible immediately; surface the loading state without blocking back.

Reproduce-first plan (failing tests, per §6.T.1 / operator mandate)

- CLIENT: a VM test where the search flow is slow/throws SocketTimeoutException → assert (1) the VM renders Error+Retry (not stuck Streaming/Empty), and (2) BackClick

during an in-flight search CANCELS the search + emits Back. Currently FAILS (no cancellation) → reproduction.
- ENGINE: a Go test where the yts upstream is slow → the search handler must return within a request-context deadline < the client timeout. Currently FAILS if unbounded → reproduction.
- Then fix root causes A + B, code review, rebuild, retest, validate (no false results, no bluff).